Human Development Index and PCA

Archer Yang

Outline

The steps to carry out PCA are:

1. The centering and scaling of the data
2. Carry out data reduction
3. Optionally, undo the centering and scaling, likely using a limited number of PCs

We'll discuss each of these steps in order through the following case study.

Data Description

PCA has been the main method available for the development of indexes, which are otherwise a hit-or-miss ad hoc undertaking.

The country-level Human Development Index (HDI) (https://hdr.undp.org) from United Nations Development Programme, which has been published since 1990 and is very extensively used in development studies, has very similar coefficients on similar indicators, strongly suggesting it was originally constructed using PCA.

The original data from 2021 can be downloaded here https://hdr.undp.org/sites/default/files/2021-22_HDR/HDR21-22_Statistical_Annex_HDI_Table.xlsx. The data consists of $N = 191$ nations. There are four variables $(D = 4)$ recorded for each nation:

- A long and healthy life: **(LE)** Life expectancy at birth (years)
- Education:
  - **(MYS)** Mean years of schooling (years)
  - and **(EYS)** Expected years of schooling (years)
- A decent standard of living: **(GNI)** GNI per capita (GNI = GDP+Net, in dollars)

First we load the data.

```
source("biplot.R", chdir = TRUE)

# load data
library(readr)
data.source<-paste0("https://raw.githubusercontent.com/",
"mcgillstat/regression/main/data/HDI.csv")

dat1 <- read_csv(data.source, col_names = TRUE)

# convert it to a data.frame
```

```r
HDI = data.frame(dat1)
# use abbreviated column names
colnames(HDI) <- c("HDI_rank","Country","HDI","LE","MYS","EYS","GNI")
# add row names to the dataset
rownames(HDI) <- HDI$Country

# show the first 10 observations
print(HDI[1:10,-c(1,2)])
```

```
##                           HDI   LE  MYS  EYS   GNI
## Switzerland             0.962 84.0 16.5 13.9 66933
## Norway                  0.961 83.2 18.2 13.0 64660
## Iceland                 0.959 82.7 19.2 13.8 55782
## Hong Kong, China (SAR) 0.952 85.5 17.3 12.2 62607
## Australia               0.951 84.5 21.1 12.7 49238
## Denmark                 0.948 81.4 18.7 13.0 60365
## Sweden                  0.947 83.0 19.4 12.6 54489
## Ireland                 0.945 82.0 18.9 11.6 76169
## Germany                 0.942 80.6 17.0 14.1 54534
## Netherlands             0.941 81.7 18.7 12.6 55979
```

Step 1. Centering and scaling

When we center the data, we take each column, corresponding to a particular variable, and subtract the mean of that column from each value in the column.

Scaling the data is optional. If the range of the variables (which, recall, are in the columns) are approximately the same, one generally does not scale the data. However, if some variables have much larger ranges, they will dominate the PCA results. You may want this to happen, or you may not, and in many cases it is wise to try different scaling options.

Step 2. Data reduction

Now we are ready for the actual PCA process.

Using prcomp

We use the `R` function `prcomp` to perform PCA (with data centering and scaling).

```r
# Perform PCA
pca_result = prcomp(~LE + MYS + EYS + GNI, data = HDI, center = T, scale = T)
# show the structure of the PCA result
str(pca_result)
```

```
## List of 6
##  $ sdev    : num [1:4] 1.778 0.632 0.5 0.434
##  $ rotation: num [1:4, 1:4] -0.5151 -0.507 -0.5009 -0.4761 0.0908 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : chr [1:4] "LE" "MYS" "EYS" "GNI"
```

```
##   .. ..$ : chr [1:4] "PC1" "PC2" "PC3" "PC4"
##  $ center  : Named num [1:4] 71.31 13.53 8.98 20249.09
##   ..- attr(*, "names")= chr [1:4] "LE" "MYS" "EYS" "GNI"
##  $ scale   : Named num [1:4] 7.65 2.93 3.17 21825.26
##   ..- attr(*, "names")= chr [1:4] "LE" "MYS" "EYS" "GNI"
##  $ x       : num [1:191, 1:4] -3.16 -3.21 -3.28 -3.04 -3.42 ...
##   ..- attr(*, "dimnames")=List of 2
##   .. ..$ : chr [1:191] "Switzerland" "Norway" "Iceland" "Hong Kong, China (SAR)" ...
##   .. ..$ : chr [1:4] "PC1" "PC2" "PC3" "PC4"
##  $ call    : language prcomp(formula = ~LE + MYS + EYS + GNI, data = HDI,      center = T, sca
##  - attr(*, "class")= chr "prcomp"
```

`str(pca_result)` shows the structure of `pca_result`, the object that holds the PCA results:

`pca_result$center` The values used for centering. There are D values.

$$(\bar{x}_1, \cdots, \bar{x}_D)$$

where

$$\bar{x}_d = \frac{1}{N}\sum_{n=1}^{N} x_{nd} \qquad d = 1, \cdots, D$$

is the d-th column mean of the data matrix.

```
# column mean of the original data
pca_result$center
```

```
##          LE         MYS         EYS         GNI
##    71.31466    13.53037     8.98377 20249.09424
```

`pca_result$scale` The values used for centering. There are D values.

$$(\mathrm{sd}_1, \cdots, \mathrm{sd}_D)$$

where

$$\mathrm{sd}_d = \sqrt{\frac{1}{N-1}\sum_{n=1}^{N}(x_{nd} - \bar{x}_d)^2}$$

is the d-th column standard deviation of the data matrix.

```
# column standard deviation of the original data
pca_result$scale
```

```
##           LE          MYS          EYS
##     7.646519     2.926039     3.173193
##          GNI
## 21825.264062
```

`pca_result$sdev` The standard deviations of the principal components, square roots of eigenvalues.

$$\sqrt{\lambda_1}, \cdots, \sqrt{\lambda_D}$$

These are used in the construction of a scree plot (coming up next). The length of `pca$sdev` is equal to D, the number of variables in the data set.

```
# square root of the eigenvalues
pca_result$sdev

## [1] 1.7779651 0.6322887 0.5003078 0.4344458
```

`pca_result$rotation` These are the **loadings**, i.e. the eigenvectors, stored *by column* in a square matrix with dimensions $D \times D$

$$\boldsymbol{B} = \begin{pmatrix} | & | & & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 & \cdots & \boldsymbol{b}_D \\ | & | & & | \end{pmatrix} \in \mathbb{R}^{D \times D}$$

```
# loading matrix, i.e. eigenvectors
loadings <- pca_result$rotation
rownames(loadings) <- rownames(pca_result$rotation)
round(loadings, 2)

##       PC1   PC2   PC3   PC4
## LE  -0.52  0.09 -0.58  0.62
## MYS -0.51 -0.44 -0.33 -0.66
## EYS -0.50 -0.41  0.70  0.30
## GNI -0.48  0.80  0.24 -0.28
```

`pca_result$x` A key part is `pca_result$x`, which holds the **PCA scores**. The matrix saves the coordinates of projection along each principal component. The n-th row of `pca_result$x` is

$$(\boldsymbol{B}^\top \boldsymbol{x}_n)^\top = \begin{pmatrix} \boldsymbol{b}_1^\top \boldsymbol{x}_n \\ \boldsymbol{b}_2^\top \boldsymbol{x}_n \\ \\ \boldsymbol{b}_D^\top \boldsymbol{x}_n \end{pmatrix}^\top = \begin{pmatrix} \boldsymbol{b}_1^\top \boldsymbol{x}_n, & \boldsymbol{b}_2^\top \boldsymbol{x}_n & , \cdots , & \boldsymbol{b}_D^\top \boldsymbol{x}_n \end{pmatrix} = (z_{1n}, z_{2n}, \cdots, z_{Dn})$$

```
# score matrix, only first 10 rows are shown
scores <- pca_result$x
round(scores, 2)[1:10, ]

##                            PC1   PC2   PC3   PC4
## Switzerland              -3.16  0.78  0.31  0.22
## Norway                   -3.21  0.55 -0.05 -0.29
## Iceland                  -3.28 -0.03 -0.05 -0.36
## Hong Kong, China (SAR) -3.04  0.74 -0.32  0.06
```

```
## Australia               -3.42 -0.39 -0.72 -0.67
## Denmark                 -3.08  0.30 -0.02 -0.49
## Sweden                  -3.12  0.05 -0.37 -0.48
## Ireland                 -3.28  1.03 -0.22 -0.82
## Germany                 -2.78  0.19  0.42  0.01
## Netherlands             -2.95  0.19 -0.18 -0.45

dim(scores)

## [1] 191   4
```

For example, for n-th obervation $\boldsymbol{x}_n$, we have

$$\boldsymbol{x}_n = (\text{LE}_n, \text{MYS}_n, \text{EYS}_n, \text{GNI}_n)^\top$$

We can compute the PC1 score z_{1n}, which represent the coordinate of the projected point along the direction $\boldsymbol{b}_1$.

$$z_{1n} = \boldsymbol{b}_1^\top \boldsymbol{x}_n = -0.52 \times \text{LE}_n - 0.51 \times \text{MYS}_n - 0.50 \times \text{EYS}_n - 0.48 \times \text{GNI}_n$$

where $\boldsymbol{b}_1$ represent the coefficients for the linear combination of variables that define the score. One coefficient value for each of the original variables.

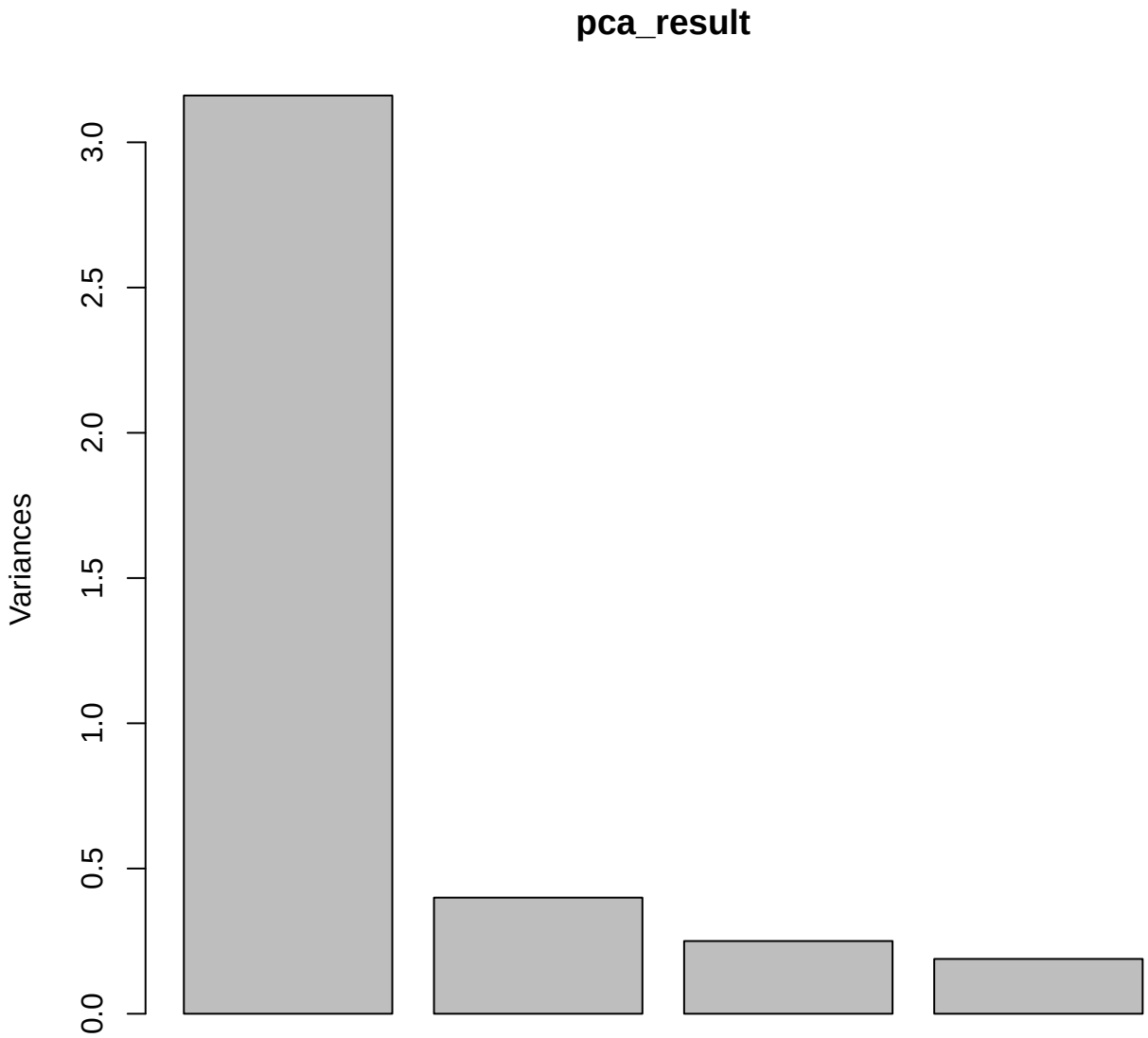
- z_{1n} provides information on how well aligned vector $\boldsymbol{b}_1$ is with the original vector $\boldsymbol{x}_n$.
- A larger coefficient of $\boldsymbol{b}_1$ indicate more influence of that corresponding variable on the PC score.

You can use similar ways to compute score z_{2n}, z_{3n} and z_{4n}

PCA Scree plot

A scree plot displays how much variation each principal component captures from the data. These values are simply the square of `pca$sdev`, i.e. $\lambda_1, \cdots, \lambda_D$ and can be plotted by calling `plot` on an object of class `prcomp`.

```
plot(pca_result)
screeplot(pca_result)
```



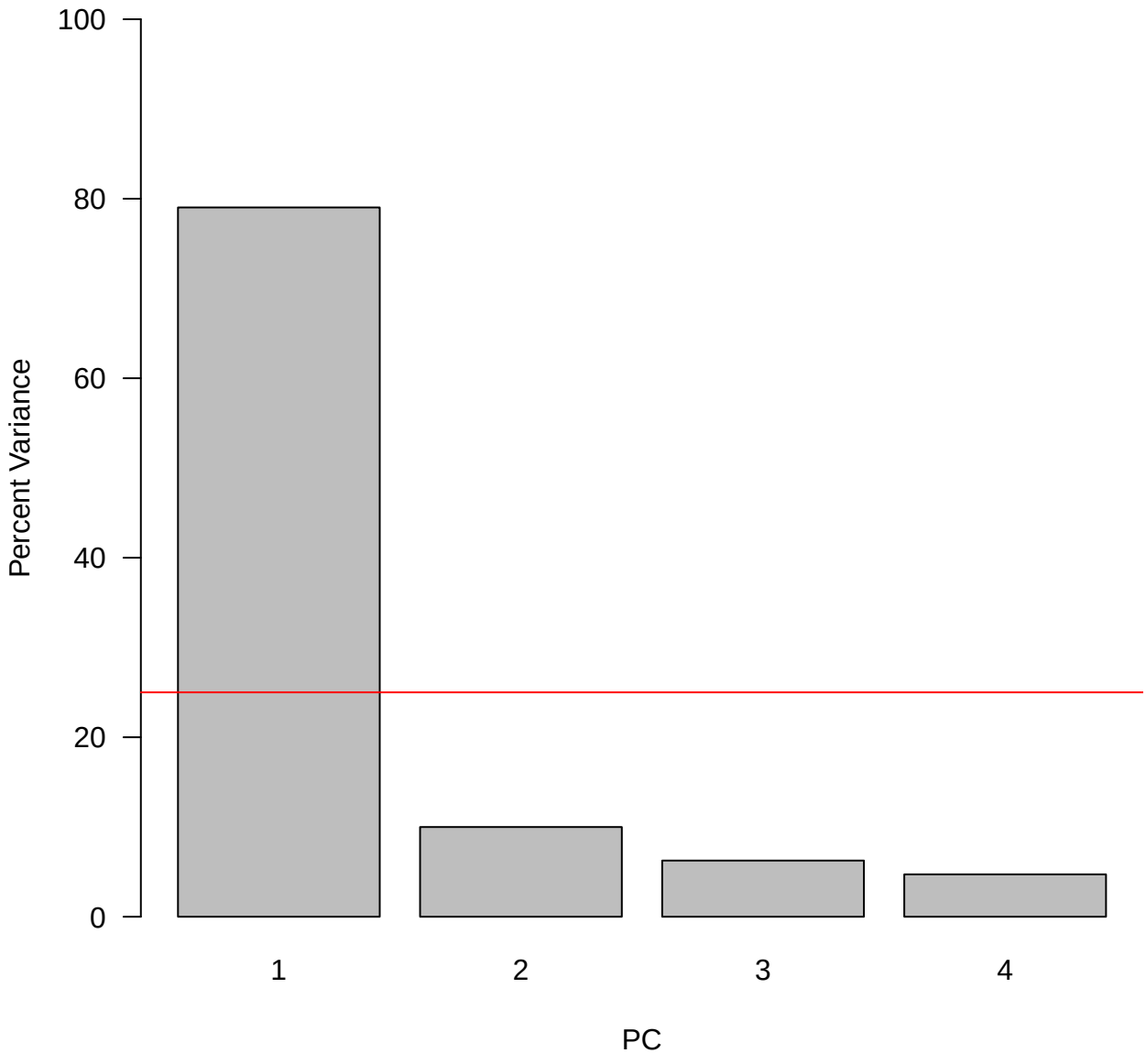
PC1 captures the most variation, PC2 the second most, and so on. We can also generate a scree plot with normalized variances. We will also add a line indicating the amount of variance each variable would contribute if all contributed the same amount. This is just a horizontal line drawn at $1/D$, where D is the number of variables (or PCs).

The instrisic dimension of the data is small if it is a steep curve, followed by a bend, and then a straight line. Can you imagine a scenario when the curve in scree plot is a flat line?

```
# to get normalized the variance

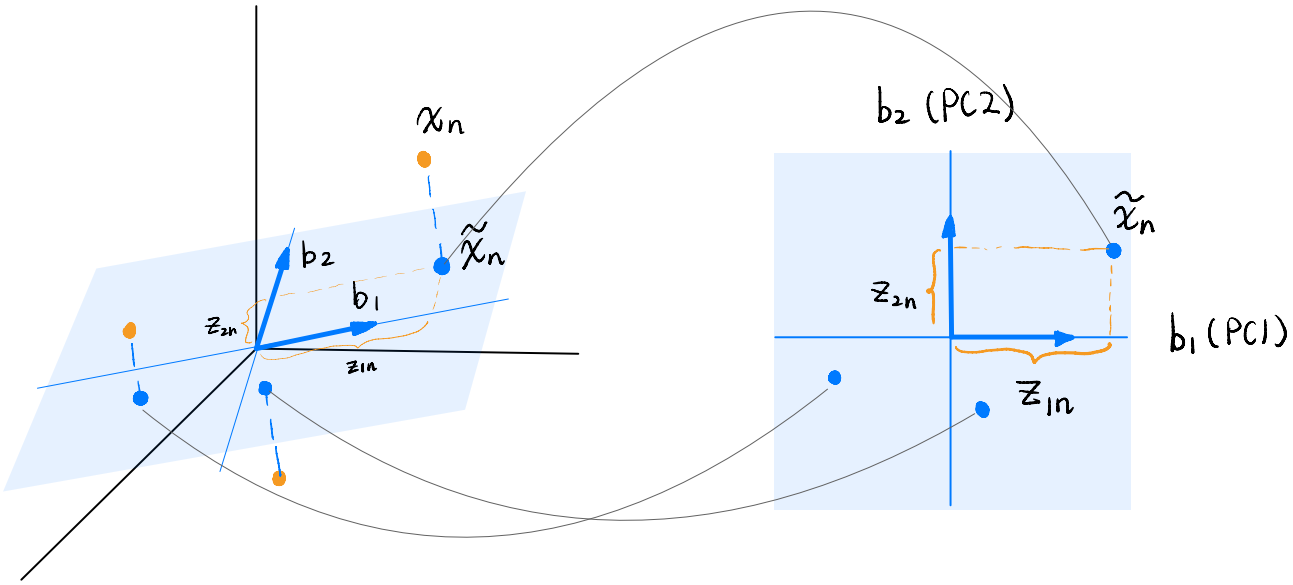
sd <- pca_result$sdev
```

```r
var <- sd^2
varPercent <- var/sum(var) * 100
barplot(varPercent, xlab="PC", ylab="Percent Variance",
names.arg=1:length(varPercent),
las=1, ylim=c(0, 100), col="gray")
abline(h = 1/4 * 100, col="red")
```



PCA score plot

The **score plot** plots the projected data in a new coordinate system, where the first component $\boldsymbol{b}_1$ and the second component $\boldsymbol{b}_2$ form the basis.

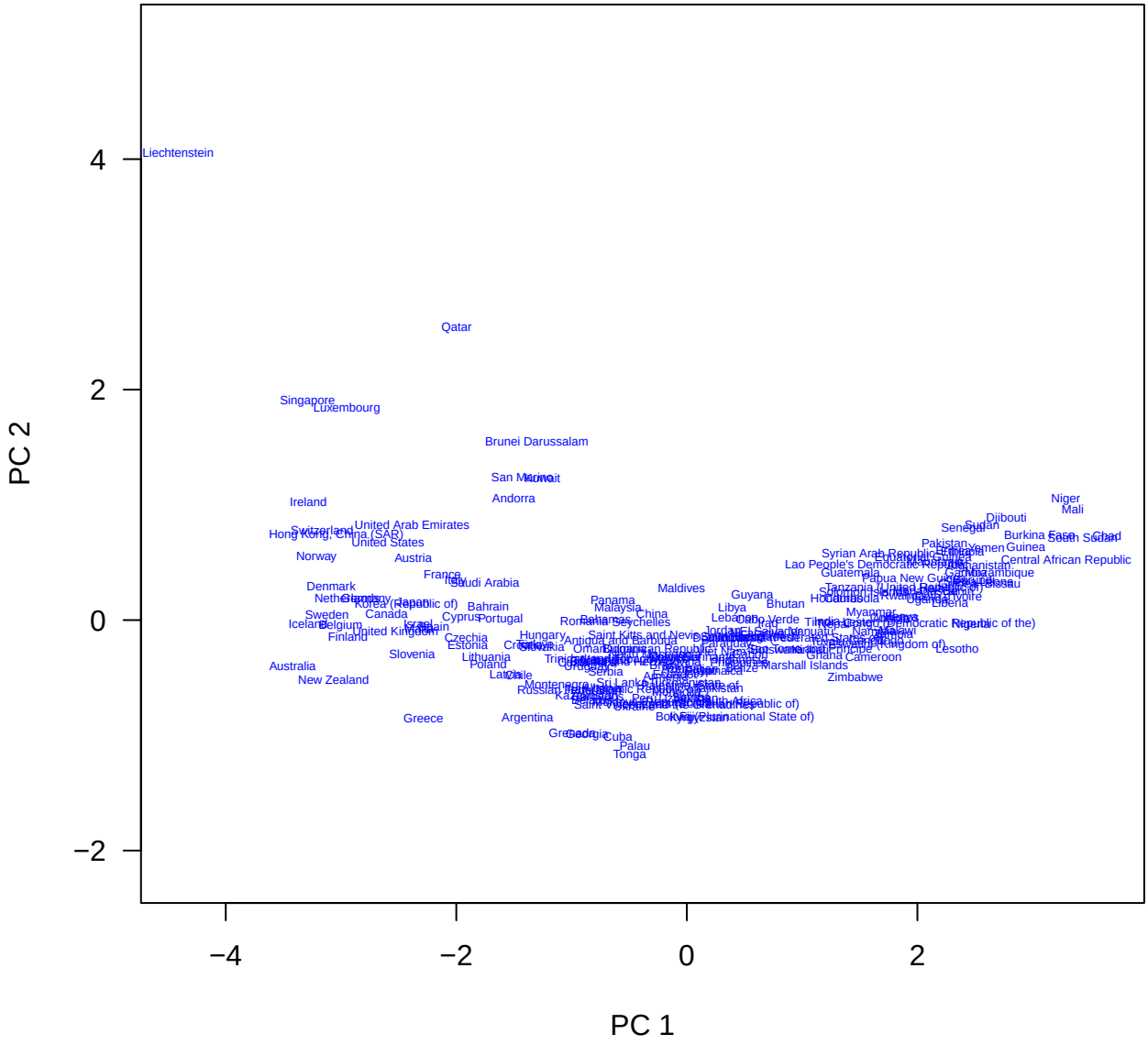


The two entries $(\boldsymbol{b}_1^\top \boldsymbol{x}_n, \boldsymbol{b}_2^\top \boldsymbol{x}_n)$ of each row of the matrix below represents the x- and y- coordinates of a projected observation in the score plot.

$$\begin{pmatrix} z_{11} & , z_{21} \\ z_{12} & , z_{22} \\ \vdots & \vdots \\ z_{1N} & , z_{2N} \end{pmatrix} = \begin{pmatrix} \boldsymbol{b}_1^\top \boldsymbol{x}_1 & , \boldsymbol{b}_2^\top \boldsymbol{x}_1 \\ \boldsymbol{b}_1^\top \boldsymbol{x}_2 & , \boldsymbol{b}_2^\top \boldsymbol{x}_2 \\ \vdots & \vdots \\ \boldsymbol{b}_1^\top \boldsymbol{x}_N & , \boldsymbol{b}_2^\top \boldsymbol{x}_N \end{pmatrix}$$

This link also gives a good explanation https://stats.stackexchange.com/questions/222/what-are-principal-component-scores.

```
plot(scores[, 1], scores[, 2], xlab = "PC 1", ylab = "PC 2", type = "n", asp = 1,
    las = 1)
text(scores[, 1], scores[, 2], rownames(scores), col = "blue", cex = 0.4)
```

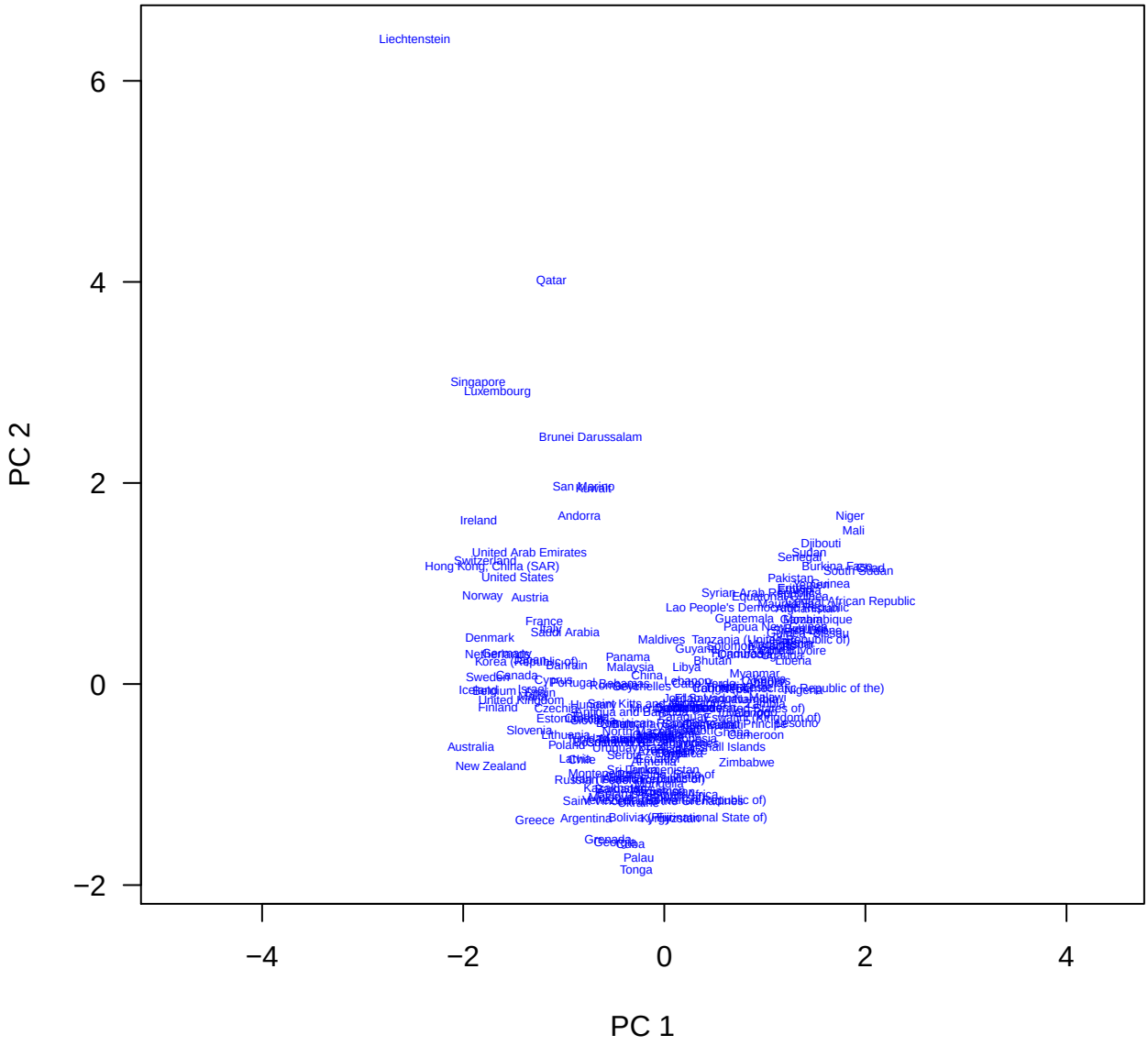


Interpretation: If the first two components account for most of the variance in the data, you can use the score plot to assess the data structure and detect clusters, outliers, and trends. Groupings of data on the plot may indicate two or more separate distributions in the data.

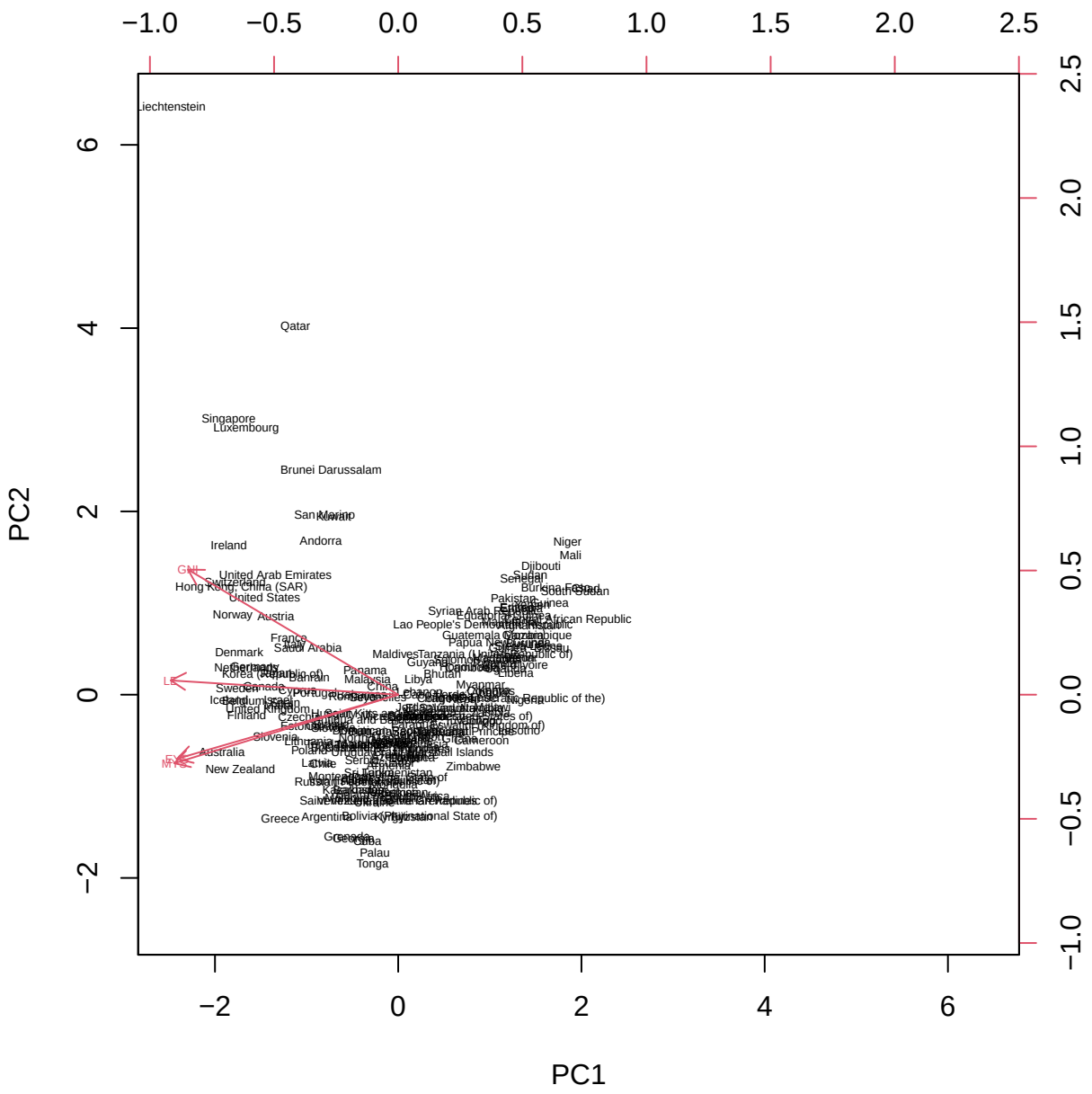
People more often use the **scaled score plot**, which scales the coordinates of PCs by their standardard deviation. This makes the distances along x- and y- axis comparable.

$$\begin{pmatrix} \frac{z_{11}}{\mathrm{sd}(z_1)} & , \frac{z_{21}}{\mathrm{sd}(z_2)} \\ \frac{z_{12}}{\mathrm{sd}(z_1)} & , \frac{z_{22}}{\mathrm{sd}(z_2)} \\ \vdots & \vdots \\ \frac{z_{1N}}{\mathrm{sd}(z_1)} & , \frac{z_{2N}}{\mathrm{sd}(z_2)} \end{pmatrix} = \begin{pmatrix} \frac{\boldsymbol{b}_1^\top \boldsymbol{x}_1}{\sqrt{\lambda_1}} & , \frac{\boldsymbol{b}_2^\top \boldsymbol{x}_1}{\sqrt{\lambda_2}} \\ \frac{\boldsymbol{b}_1^\top \boldsymbol{x}_2}{\sqrt{\lambda_1}} & , \frac{\boldsymbol{b}_2^\top \boldsymbol{x}_2}{\sqrt{\lambda_2}} \\ \vdots & \vdots \\ \frac{\boldsymbol{b}_1^\top \boldsymbol{x}_N}{\sqrt{\lambda_1}} & , \frac{\boldsymbol{b}_2^\top \boldsymbol{x}_N}{\sqrt{\lambda_1}} \end{pmatrix}$$

```r
scores_scaled <- t(t(scores[,1L:2L]) / pca_result$sdev[1L:2L])
plot(scores_scaled[, 1], scores_scaled[, 2],
        xlab="PC 1", ylab="PC 2", type="n",
        asp=1, las=1)
text(scores_scaled[, 1], scores_scaled[, 2],
        rownames(scores_scaled), col="blue", cex=0.4)
```



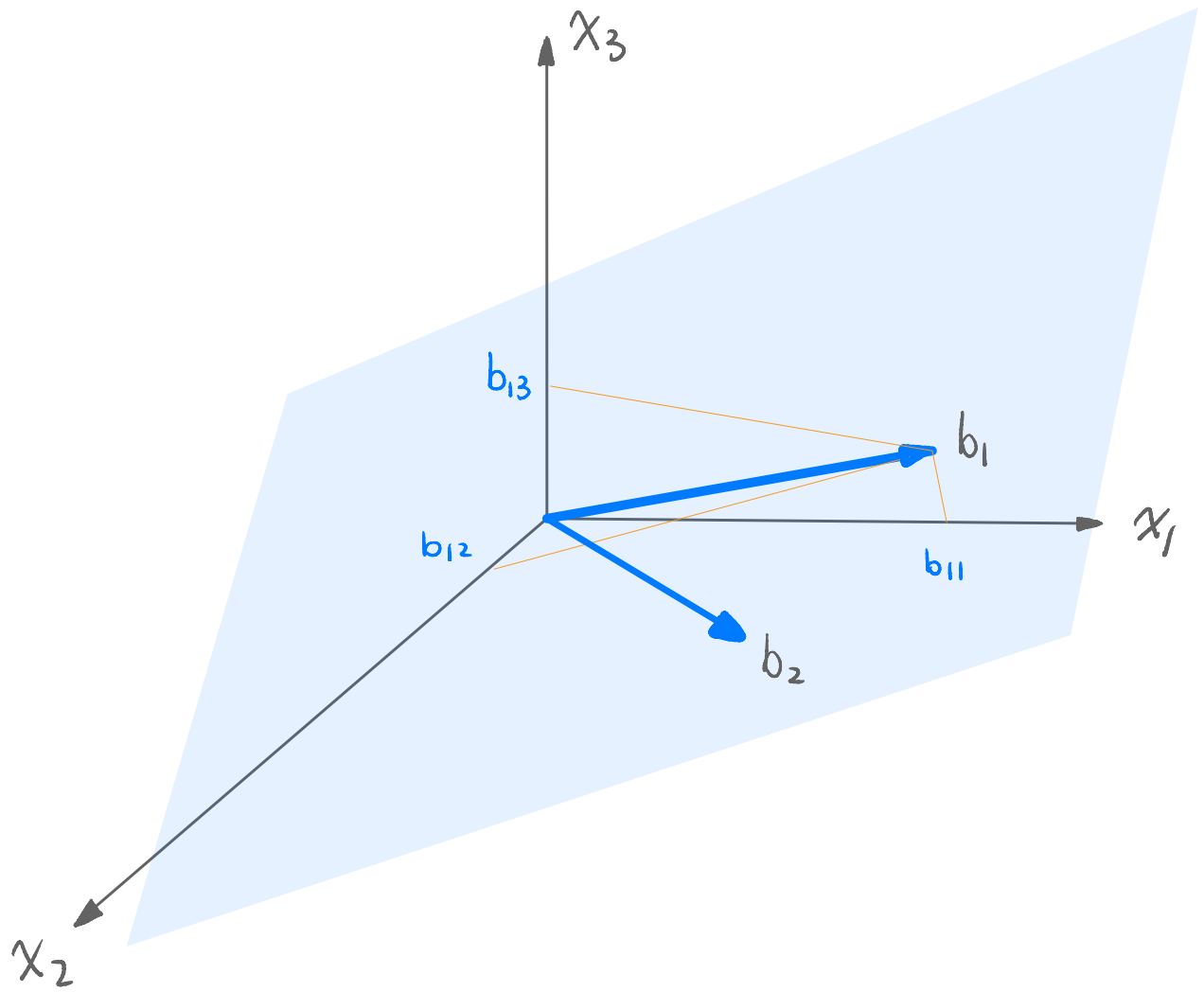

```r
# you can also use a single line command to make the scaled score plot
biplot(pca_result, cex=0.4, pc.biplot=TRUE, var.axes= FALSE)
```




```
##            PC1         PC2
## LE  -0.9159097  0.05738244
## MYS -0.9015038 -0.27609724
## EYS -0.8905337 -0.25819174
## GNI -0.8464692  0.50359070
```

PCA loading plot and biplot

The loading plot graphs the coefficients of each variable for the first component versus the coefficients for the second component.



The two entries (b_{1d}, b_{2d}) of the d-th row of the matrix below represents the x- and y- coordinates of in the loading plot. 4

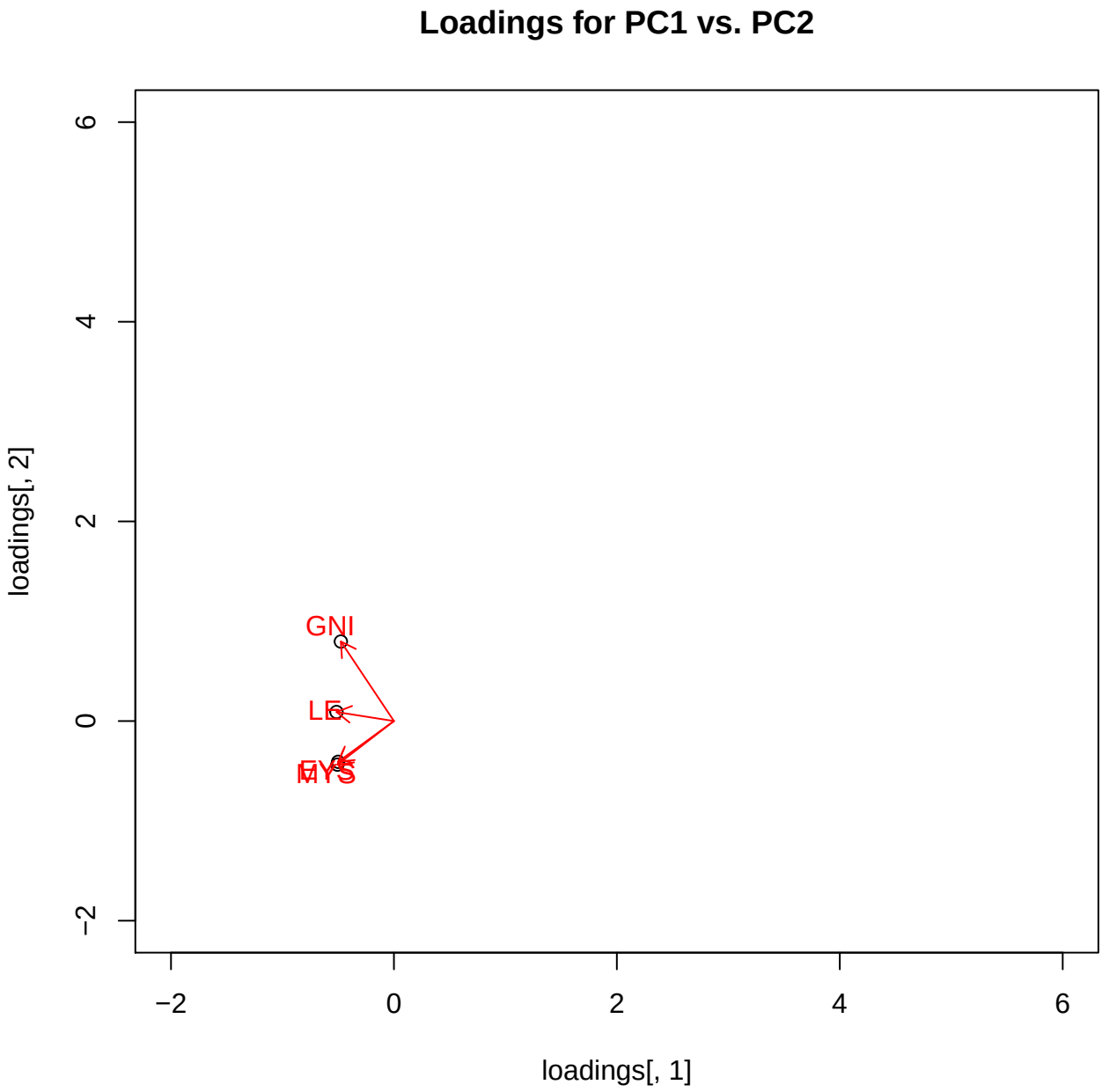
$$\begin{pmatrix} | & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 \\ | & | \end{pmatrix} = \begin{pmatrix} b_{11} & , b_{21} \\ b_{12} & , b_{22} \\ \vdots & \vdots \\ b_{1D} & , b_{2D} \end{pmatrix}$$

Interpretation: Use the loading plot to identify which variables have the largest effect on each component. Large loading indicates that the corresponding variable strongly influences the component. Evaluating the loadings can also help you characterize each component in terms of the variables.

```
plot(loadings[,1], loadings[,2],
    xlim=c(-2,6), ylim=c(-2,6),
        main='Loadings for PC1 vs. PC2')

textNudge <- 1.2
arrows(0, 0, loadings[, 1], loadings[, 2],
        length=0.1,  col="red")
```

```
text(loadings[, 1] * textNudge, loadings[, 2] * textNudge,
        rownames(loadings), col="red", cex=1)
```

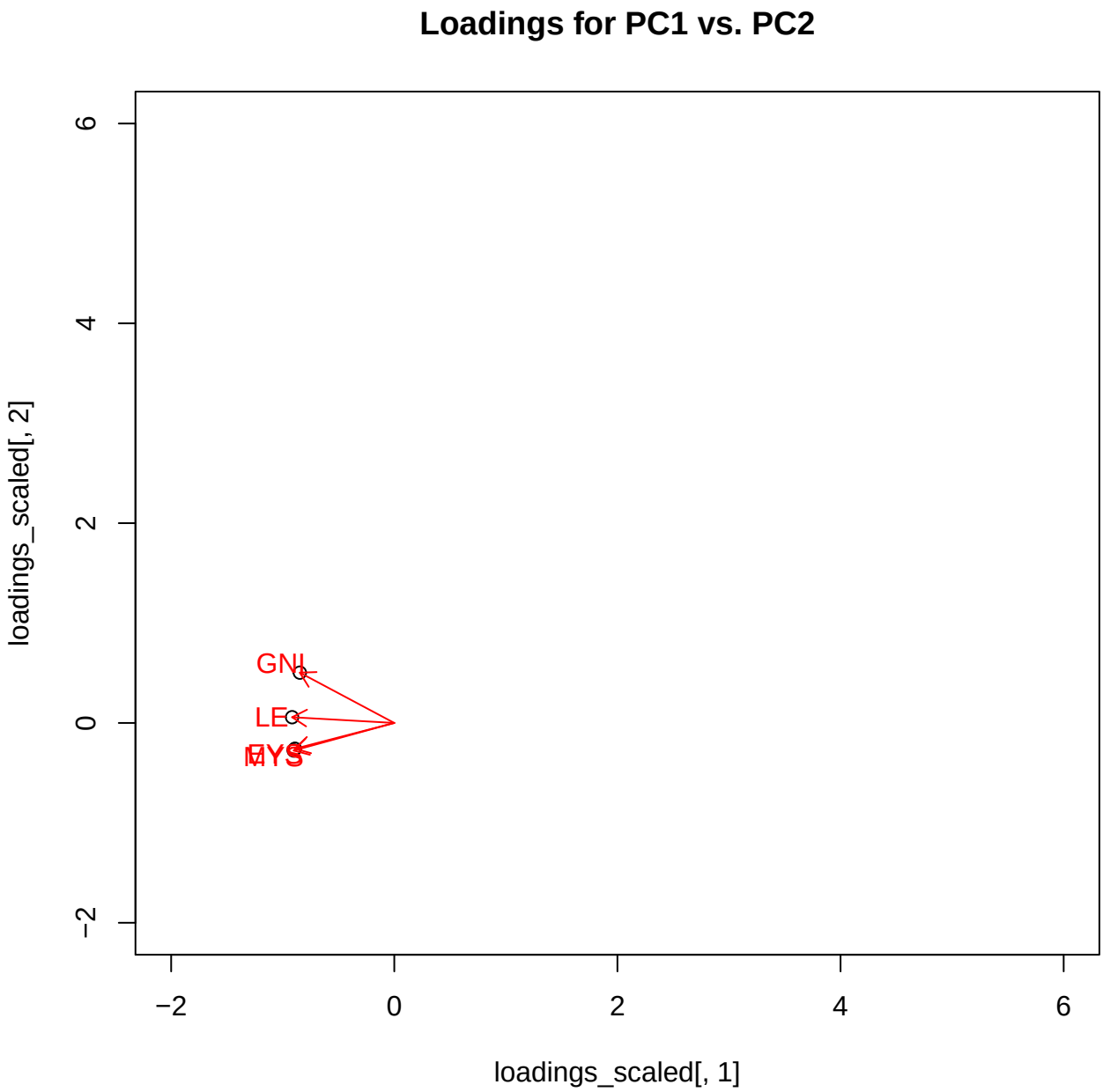



```
loadings_scaled <- t(t(loadings[,1L:2L]) * pca_result$sdev[1L:2L])

plot(loadings_scaled[,1], loadings_scaled[,2],
     xlim=c(-2,6), ylim=c(-2,6),
     main='Loadings for PC1 vs. PC2')

arrows(0, 0, loadings_scaled[, 1], loadings_scaled[, 2],
        length=0.1, col="red")
```

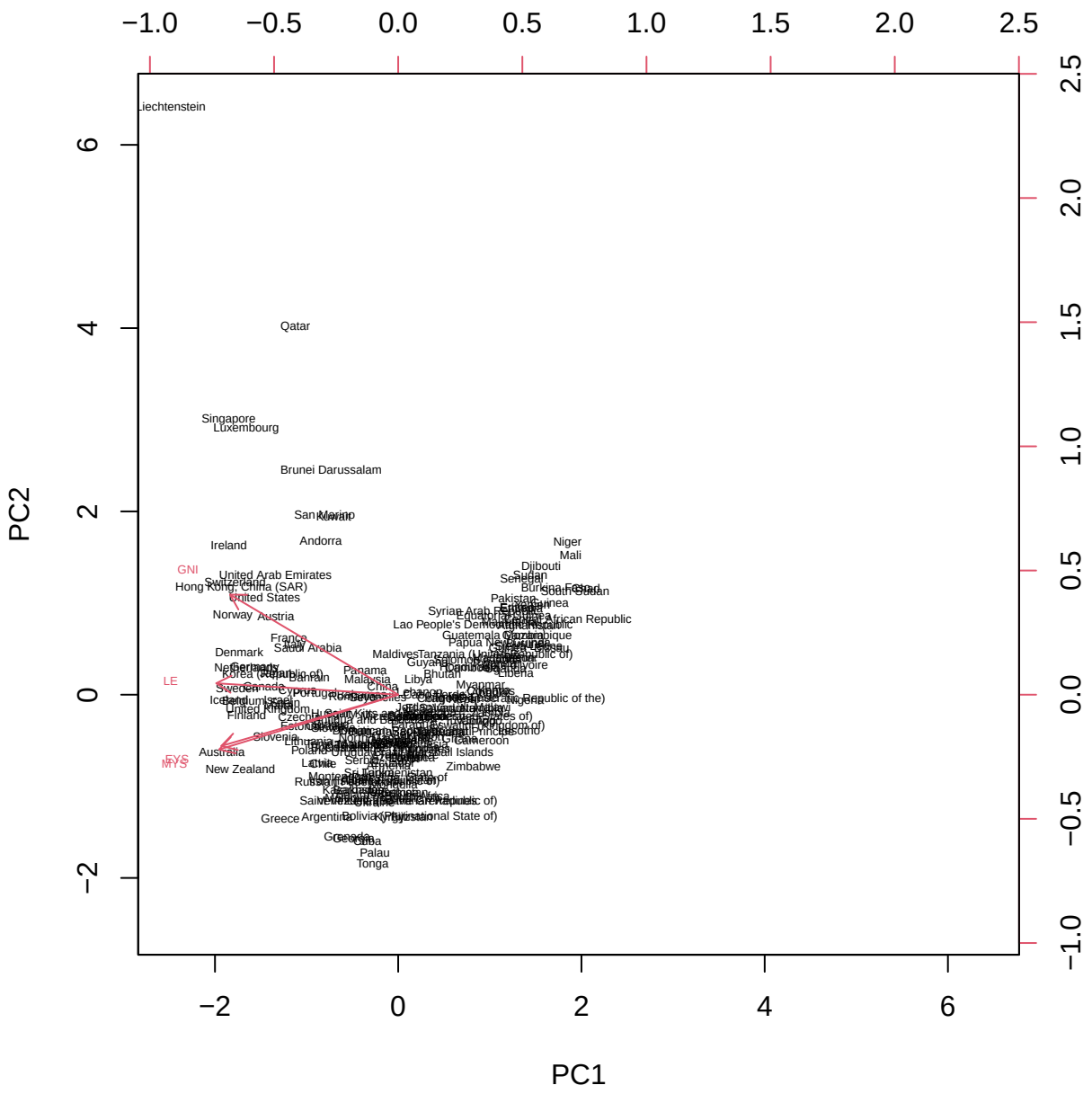
```
text(loadings_scaled[, 1] * textNudge,
        loadings_scaled[, 2] * textNudge,
        rownames(loadings_scaled), col="red", cex=1)
```



Biplot

The biplot overlays the score plot and the loading plot.

```
# you can also use a single line command to make the scaled biplot
biplot(pca_result, cex = 0.4, pc.biplot = TRUE)
```



Note that the axes on the left and bottom of the plot are for the score plot. The axes on the right and top of the plot are for the loading plot.

Step 3. Undo the centering and scaling

Sometimes it is desirable to reconstruct the original data using a limited number of PCs. If one reconstructs the original data using all the PCs, one gets the original data back. However, for many data sets, the higher PCs don't represent useful information; effectively they are noise. So using a limited number of PCs one can get back a reasonably faithful representation of the original data.

To reconstruct all or part of the original data, one starts from the object returned by `prcomp`. If one wants to use the first $M = 2$ PCs to reconstruct the data, one takes the first 2 columns of the rotation

matrix (in pca$rotation)

$$\texttt{pca\$rotation} = \boldsymbol{B} = \begin{pmatrix} | & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 \\ | & | \end{pmatrix}$$

and multiplies them by the transpose of the first $M = 2$ scores (in pca$x)

$$\texttt{t(pca_result\$x[,1:2])} = \boldsymbol{B}^\top \boldsymbol{x}_n = \begin{pmatrix} \boldsymbol{b}_1^\top \boldsymbol{x}_n \\ \boldsymbol{b}_2^\top \boldsymbol{x}_n \end{pmatrix} = \begin{pmatrix} z_{1n} \\ z_{2n} \end{pmatrix}$$

The constructed $\tilde{\boldsymbol{x}}_n$ therefore is

$$\tilde{\boldsymbol{x}}_n = \begin{pmatrix} | & | \\ \boldsymbol{b}_1 & \boldsymbol{b}_2 \\ | & | \end{pmatrix} \begin{pmatrix} \boldsymbol{b}_1^\top \boldsymbol{x}_n \\ \boldsymbol{b}_2^\top \boldsymbol{x}_n \end{pmatrix} = z_{1n}\boldsymbol{b}_1 + z_{2n}\boldsymbol{b}_2$$

In R this would be expressed for the data set as:

```
Xtilde<- pca_result$x[, 1:2] %*% t(pca_result$rotation[, 1:2])
```

where Xtilde is the approximation matrix.

We are now ready to undo the scaling, which is accomplished by dividing the columns of Xtilde by the scale factors previously employed. Once again the scale() function makes it easy to operate on the columns of the matrix.

```
Xtilde <- scale(Xtilde, center = FALSE, scale = 1/pca_result$scale)
```

Finally, we take the unscaled Xtilde and add back the values that we subtracted when centering, again using scale()

```
Xtilde <- scale(Xtilde, center =  -pca_result$center, scale = FALSE)
```

One might think that scale can handle both the unscaling and re-centering processes at the same time. This is not the case, as scale does any (un)centering first, then scales the data. We need to take care of scaling first, and then the centering second. Thus in the forward direction, scale can handle both tasks simultaneously, but in the reconstruction direction, we need to take it step-wise.

Proof of Perfect Reconstruction

If we use only a portion of the PCs, an approximation of the original data is returned. If we use all the PCs, then the original data is reconstructed.

```
Xtilde_full <- pca_result$x %*% t(pca_result$rotation)
Xtilde_full <- scale(Xtilde_full, center = FALSE, scale = 1/pca_result$scale)
Xtilde_full <- scale(Xtilde_full, center =  -pca_result$center, scale = FALSE)
```

If this process worked correctly, there should be no difference between the reconstructed data and the original data.

```r
X_raw = as.matrix(HDI[, -c(1,2,3)])
mean(Xtilde_full - X_raw)

## [1] -2.366112e-14
```

Small precision error is due to numerical inaccuracy of the program. Mission: success!

Reverse-engineering (shanzhai) the HDI Index

(Homework) Construct a homemade HDI index using PC1 and rank the nations accordingly. Compare it with the real HDI index.